

A software product line approach for developing hybrid software systems

Samuel Sepúlveda^{a,*}, Ricardo Pérez-Castillo^b, Mario Piattini^c

^a Departamento de Ciencias de la Computación e Informática (DCI), Universidad de La Frontera, Av. Francisco Salazar 01145, Temuco, 4811230, Chile

^b Faculty of Social Sciences & IT, University of Castilla-La Mancha, Avenida Real Fábrica de Sedas s/n, Talavera de la Reina, 45600, Spain

^c Departamento de Tecnologías y Sistemas de Información, Paseo de la Universidad 4, Ciudad Real, 13071, Spain

ARTICLE INFO

Keywords:

Quantum computing
Software product lines
Variability
Feature modeling
Hybrid systems

ABSTRACT

Context: Quantum computing is rapidly emerging as a transformative force in technology. We will soon increasingly encounter hybrid systems that combine quantum technology with classical software. Software engineering techniques will be required to manage the complexity of designing such systems and their reuse. **Objective:** This paper introduces preliminary ideas concerning developing quantum–classical software using a Software Product Line approach.

Method: This approach addresses the mentioned challenges and provides a feature model and a whole process to manage variability during the design and development of hybrid quantum–classical software. The usage of this approach is illustrated and discussed using an example in the logistics domain.

Results: The preliminary insights show the feasibility and suitability of applying the proposed approach to develop complex quantum–classical software.

Conclusions: The main implication of this research is that it can help to manage complexity, maximize the reuse of classical and quantum software components, and deal with the highly changing technological stack in the current quantum computing field.

1. Introduction

Quantum Computing (QC) emerges not as a new traditional programming paradigm but as a revolutionary computational architecture with an innovative approach. However, many current Software Engineering (SE) concepts and techniques designed to be platform-independent will remain relevant, although they will need adaptations to the unique characteristics of QC [1]. Regrettably, current quantum software development is primarily focused on low levels of abstraction, evidencing the need for engineering support at higher levels. Although such higher levels of abstraction still need to be fully developed for quantum computing, it is essential to address current challenges to pave the way for future innovations [2].

Quantum Software Engineering (QSE), emerging with QC, is dedicated to developing large-scale quantum software. Barbosa [3] note that this specialization involves designing and coding applications unique to QC, addressing its specific challenges. Zhao [4] defines QSE as applying SE principles to create and maintain quantum software, which includes managing the distinct aspects of quantum computing like superposition and entanglement. Despite advances in quantum software tools, as mentioned by Alexeev et al. [5], a complete set of tools for developing quantum software is still emerging [6]. Quantum programming and algorithm creation complexity require rigorous software

development techniques [7]. Therefore, QSE needs specialized methods and tools to effectively address these quantum-specific challenges and ensure the reliability of quantum systems [6].

Currently, “quantum applications are, in most cases, hybrid” [8]. The Software Engineering Institute (SEI) stated that more research is needed on how to “build hybrid algorithms that take advantage of the best quantum and classical algorithms working together” [2]. The development lifecycle of these hybrid software systems involves several challenging factors and many decision-making points [9,10]. Thus, a high variability must be managed during the whole lifecycle, which implies answering many questions. For example, should a given functionality be supported by classic software or the quantum counterpart? What kind of quantum software should be used, gate-based or annealing? What quantum algorithm might be used to solve that functionality? What quantum hardware should we choose, and what are their constraints?

In order to deal with this challenge, this research proposes to use a Software Product Line (SPL) approach to cope with the variability in hybrid software systems. The SPL approach considers intensive software systems that share a standard set of features to meet the specific needs of a market segment [11]. SPLs are developed from a standard

* Corresponding author.

E-mail address: samuel.sepulveda@ufrontera.cl (S. Sepúlveda).

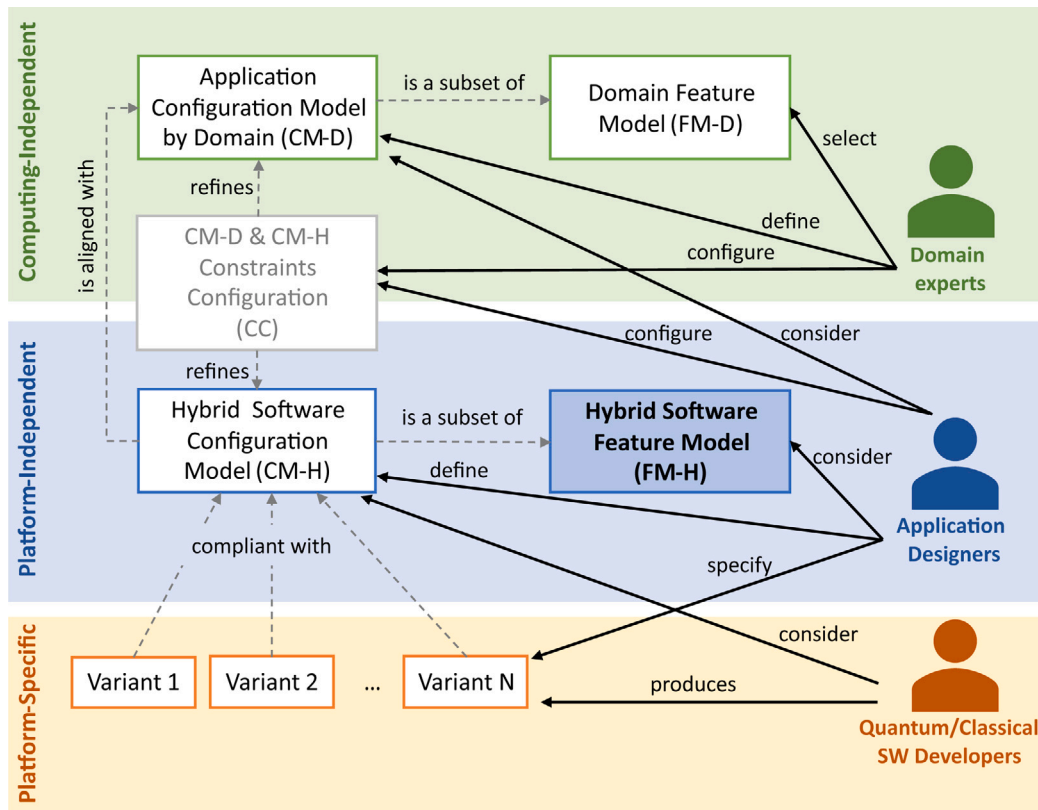


Fig. 1. SPL stages, domain and application engineering.

set of pre-established building blocks, including their architecture, software components, and products [12].

A key concept in developing SPLs is variability, which gives SPLs the flexibility and diversity necessary to differentiate products [13,14]. It is expected to describe SPLs and manage their variability with the help of a Feature Model (FM), whose structure is a kind of tree in which its root node represents the product family. The features are organized along the tree [15]. Concrete software products emerge from the assembly of these features. Moreover, some features depend on others for their inclusion, and the presence of certain features may preclude the selection of others. The main contribution of this paper is an FM to manage the inherent variability during the development of hybrid software systems.

The primary implication of this research is its facilitation of the customization and scalability of hybrid systems, alongside offering a systematic method to capture the evolving landscape of quantum computing capabilities. Consequently, this research establishes an initial framework for hybrid software development projects, allowing the decomposition of technological and domain-specific challenges. This decomposition will be formalized using the Model Driven Engineering (MDE) approach.

The remainder of this paper is structured as follows. Section 2 summarizes the main concepts related to the proposal. Sections 3 and 4 present the related work and the proposal, respectively. Sections 5 and 6 present an example of the proposal's application and discusses the main results. Finally, Section 7 presents the conclusions and further research.

2. Background

This section reports on the main concepts that support the proposal, including SPLs, variability, FMs, quantum computing, and quantum software engineering.

2.1. Software product lines

An SPL is a set of similar software products created from reusable artifacts in a specific application domain context [16]. Such products share common characteristics to satisfy the specific needs of a particular market segment [11]. The main objective of SPLs is to develop a family of products based on a set of common elements while providing mechanisms to manage the differences of each particular product [17].

Software development through SPLs involves two stages: domain engineering and application engineering [18,19]. The domain engineering stage is about identifying commonalities and differences between product family members and implementing a set of shared software artifacts. The application engineering stage involves constructing individual products using a subset of these shared software artifacts. Fig. 1 shows the domain and application engineering stages and their respective interactions, highlighting the importance of shared software artifacts.

Following, we show how SPL-based products differ from commercial-off-the-shelf (COTS) products. Likewise, SPL software development offers clear advantages over COTS products in terms of customization, control, and evolution. SPL enables organizations to manage product variability efficiently through a common architecture, facilitating the creation of tailored solutions for different markets or specific customers [20]. In contrast, COTS products are designed for a broad market and offer fewer customization options. Also, SPL allows organizations to maintain complete control over product evolution, enabling continuous improvements and updates, which is limited in COTS products that rely on external providers for upgrades [21]. This flexible approach also significantly reduces adaptation costs, as derived products share a joint base [22].

Although the approach used for SPL development has brought benefits, according to different authors, we can mention the reduction in development costs, quality improvement, customized software, and the reduction of development time, among others [16,18,23]. For the

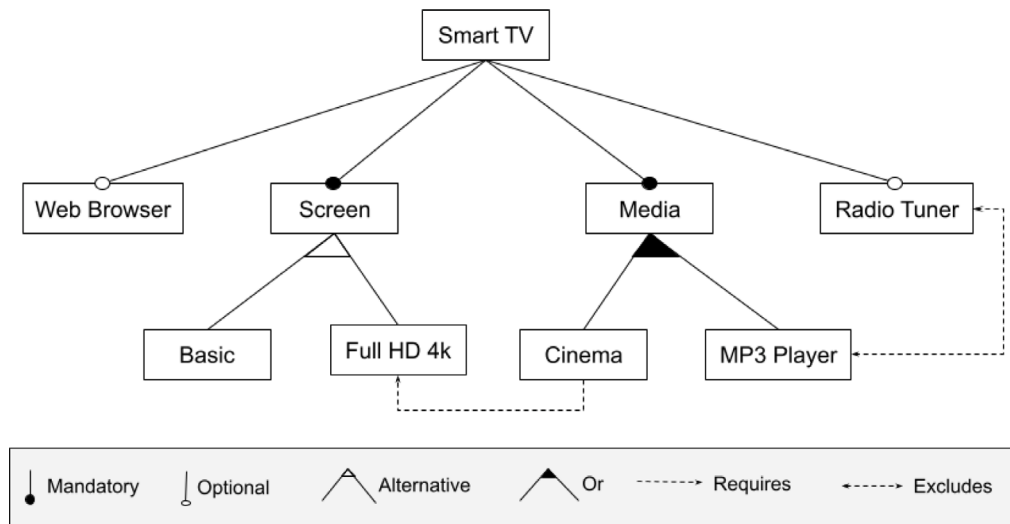


Fig. 2. Example of SPL, variability and feature model.

successful implementation of an SPL, it is necessary to pay attention to certain aspects not present in other forms of software development, such as the planned reuse of components [24]. A complete analysis of the evolution of SPLs is presented in [25].

2.2. Variability modeling

One of the distinctive aspects of SPL is variability. It provides SPLs the flexibility required to diversify and differentiate products [26]. It can be defined as the ability of an SPL to be subject to change, customization, configuration, or extension for use in a specific context. Variability is introduced in SPL through the definition of reusable artifacts (e.g., architectures or components) included as part of the definition of a product family and that, depending on their inclusion/exclusion in each final product, allow different products to arise [17].

Although correct variability management allows one to maximize the return on investment for constructing and maintaining products in an SPL, this management is a complex task whose main challenge lies in the relationship between each SPL's different "features" [27].

Different authors have proposed models to manage the variability of an SPL, most of them based on the FODA analysis method [28], which consists of features and relationships that extend graphically in the form of a tree.

2.3. Feature model

It is expected to describe an SPL and manage its variability with the help of an FM [29]. An FM is a tree-like structure whose root node represents the application product family. The SPL features are organized along the tree. The leaves of this FM represent individual features or groups of features, which can be assembled to produce particular software products [15].

Fig. 2 shows a simplified SPL and its FM inspired by the electronics industry, particularly Smart TVs. This FM shows that every Smart TV must have a Screen and Media resources. However, it may or may not have a Web Browser or Radio Tuner. Having an FM, a customer may ask for a specific configuration, i.e., a set of features. For example, enabling the set of features Smart TV, Screen, Basic, Media, MP3 Player form a valid configuration. Therefore, a set of enabled features is a set of features whose grouping is valid. Note that mandatory features must be present in a valid configuration if their parents are present. In contrast, the configuration Smart TV, Screen, Media, MP3 Player is

not valid since no sub-feature of Screen is selected. Note that all valid configurations must include the whole Smart TV (root node).

A valid configuration must include the root feature. In addition to this concept, the FM allows the following relationships among its features: mandatory, optional, alternative, and disjunction.

- **Mandatory.** A child feature has a mandatory relationship with its parent when the child is included in all products in which its parent feature appears. For instance, every Smart TV in our example must have a Screen.
- **Optional.** A child feature has an optional relationship with its parent when the child can be included optionally in all products in which its parent feature appears. In the example, a Smart TV may optionally include a Web Browser.
- **Alternative.** A set of child features has an alternative relationship with the parent when only one feature of the children can be selected if its parent feature is part of the product. For example, a Smart TV may provide Basic or Full HD 4k screens.
- **Disjunction (Or).** A set of child features has an or-relationship with its parent when one or more of them can be included in the products in which its parent feature appears. In Fig. 2, Cinema or MP3 Player can be selected whenever Media is selected.

In addition to the parental relationships within features, an FM can contain cross-tree constraints between features. These are typically in the form of require or exclude.

- **Require.** If a feature A requires a feature B, the inclusion of A in a product implies the inclusion of B, too. In the example of Smart TV, Cinema requires the inclusion of Full HD 4k.
- **Exclude.** If feature A excludes feature B, neither feature can be part of the same product. In our example, a Radio Tuner and MP3 Player are incompatible.

From academia and industry, many proposals use FMs to represent and manage SPL variability from 1990 to the present [28,30–32]. Several ISO/IEC standards on SPL have been already developed. ISO/IEC 26550:2015 Software and Systems Engineering, a product line engineering and management reference model [33]. ISO/IEC 26580:2021 Software and Systems Engineering, which includes methods and tools for the feature-based approach to software and systems product line engineering [34].

In [29,35], the authors present a first synthesis of the challenges posed by using FMs, including the lack of an adequate conceptual basis, proposals focused on functional characteristics, and most proposals

have not demonstrated their feasibility in real environments, among others. A more recent review shows that many proposals have emerged that address the errors associated with FM. In [36], authors present how defect prediction models can identify faulty features using machine learning techniques. A generic algorithm for different FM anomalies is proposed by [37]. A survey with practitioners to analyze missing capabilities in software variability management tools and compare these results with the scientific literature through a systematic mapping study is presented by [38]. The author's findings show that many tools need more essential qualities, such as interoperability, collaborative work, code generation, scalability, impact analysis, and testing.

FMs are also used in other domains, such as cyber-physical systems [39] and autonomous underwater vehicles [40]. A complete view of the integration of FMs is presented in [41].

2.4. Quantum computing

Quantum computing leverages quantum phenomena such as superposition and entanglement for computational operations. Emerging from early 20th-century quantum mechanics theories, QC has evolved from theory to a technology poised to revolutionize various fields due to its ability to solve complex problems more quickly than classical computers [42].

Unlike classical bits, which exist in states of 0 or 1, qubits can exist in multiple states simultaneously through quantum superposition. This unique feature enables quantum computers to process many possibilities simultaneously, significantly enhancing their computational power [43]. Qubits can represent numerous states simultaneously until measured when they collapse to a specific state, allowing for parallel computations [44]. Quantum entanglement, where qubits' states become interdependent regardless of distance, is crucial for new forms of quantum communication and enhances the efficiency of quantum calculations [45].

Quantum interference manipulates the probabilities of quantum states to amplify correct probabilities and cancel incorrect ones, ensuring accurate results in quantum computations [46]. Qubits are error-prone due to environmental interactions, necessitating advanced error correction techniques. The current challenge in manufacturing large-scale quantum computers limits the complexity of problems they can solve [47]. This challenge includes a critical issue, such as decoherence, where quantum states lose information due to environmental noise. Improving the purity of materials and controlling surface interactions can reduce this problem, leading to more reliable quantum processors capable of handling tasks like simulations and optimization. Another problem is that the quantum dots, which hold qubits, face challenges like charge noise and interface inconsistencies. Developing better materials can improve their stability and scalability, enhancing their performance in chemical simulations and cryptography. Superconducting qubits suffer from surface defects and impurities. However, innovations in fabrication methods and better substrates could extend their coherence times, allowing for more complex algorithms such as quantum error correction and machine learning. Finally, the scalability of quantum systems is limited by defects at material interfaces, which need to be addressed to build larger quantum processors that can tackle significant problems like integer factorization using Shor's algorithm.

We must consider that specific problems are inherently complex or inefficient to solve using classical computers, emphasizing the necessity of QC solutions. Classical algorithms struggle with tasks such as factoring large numbers, simulating quantum systems, and solving optimization problems, as their complexity becomes computationally prohibitive [48–51]. In this context, quantum computers, leveraging superposition and entanglement, offer significant advantages by providing exponential speedups in specific algorithms. Some examples are Shor's algorithm for factoring and the Quantum Approximate Optimization Algorithm (QAOA) for solving combinatorial optimization problems. These capabilities underscore the essential role of QC in addressing complex problems for classical systems.

2.5. Quantum software engineering

Quantum software will fully achieve and transfer all the promising QC applications to the industry. There is an increasing interest in quantum software technology [52]. Despite those advances, it should be ignored that quantum software has its peculiarities, e.g., a non-deterministic and probabilistic nature and a lack of control and abstraction structures, among others.

Due to the specific nature of quantum software, Quantum Software Engineering (QSE) has emerged as a new discipline [4,6]. QSE is essential for addressing the challenges of quantum software development. QSE aims to establish robust methodologies, tools, and processes for building reliable and scalable quantum applications [4]. The need for a systematic approach to QSE, identifying current practices and challenges, is highlighted by [53]. Additionally, integrating model-driven engineering techniques can streamline the development of quantum applications, ensuring a structured and efficient development lifecycle [54].

One of the open research areas in QSE is the development of hybrid (classical-quantum) software systems [2]. Recent advancements highlight the importance of hybrid software systems in efficiently solving computationally intensive tasks through integrated quantum and classical algorithms [55]. However, a hybrid system introduces additional challenges in terms of communication and synchronization between these components [8]. Besides, the authors emphasize that maintaining system integrity presents technical challenges. One of these challenges is error propagation, where even small mistakes in either quantum or classical components potentially disrupt the entire application. This error propagation underscores the importance of implementing robust error detection and recovery mechanisms to safeguard against such failures. Another challenge is ensuring state consistency, as quantum and classical states must remain aligned even when they operate asynchronously—a task that requires constant vigilance. Finally, the effective management of resource allocation, both quantum and classical, is crucial for maintaining system stability and optimizing performance, ensuring seamless integration between the two components.

Hybrid software systems are crucial due to current limitations in fully operational quantum computers. These systems leverage classical computers for tasks they excel at, while quantum computers address problems suited to quantum algorithms [56]. Also, the development lifecycle of these hybrid software systems involves several challenging factors and many decision-making points [9].

3. Related work

This section reviews seven significant contributions to this field, highlighting their main proposals as they address problems similar to our research, although they do not propose an SPL-based approach.

In the realm of hybrid quantum applications, [57] addresses the need for sophisticated control and data flow management by proposing dual orchestrations—workflow and topology orchestrations. Their solution includes a workflow orchestration and provisioning engine, validated through a prototype using Camunda and OpenTOSCA, introducing a Quantum Application Archive (QAA) to streamline application setup and execution.

The proposal of [58] focuses on integrating quantum computing with cloud services, addressing challenges such as hardware dependency and the error-prone nature of NISQ computers. They propose a Quantum Application as a Service (QaaS) model to optimize quantum algorithms and remove hardware dependencies, aiming to establish best practices for deploying and integrating quantum applications.

The issue of inconsistencies across various quantum computing platforms due to incompatibilities is presented by [59]. They introduce the Quantum Factory Method, which uses software engineering patterns like Factory Method and Adapter to create a single definition applicable

Table 1
Common limitations and details for related work.

Limitation	Details
Complexity	The integration and synchronization of hybrid systems introduce significant complexity. The communication between quantum and classical components and their distinct resource requirements is a recurring challenge.
Scalability	The proposals struggle with scalability, particularly when addressing more significant, complex problems. Current quantum hardware limitations, make it challenging to scale quantum applications effectively.
Latency and Performance	The interaction classical–quantum often leads to delays that degrade overall system performance, especially for real-time applications. Latency, particularly in cloud-based quantum computing, is a consistent issue.
Standardization and Interoperability	A lack of standardization across quantum platforms and tools results in fragmented ecosystems. It makes challenging to develop cross-platform solutions or switch between different quantum service providers, complicating the development and integration of quantum systems.
Unreliability and Error-Prone	Current quantum hardware's unreliability, characterized by issues like decoherence and gate errors, is a critical obstacle. These errors limit QC reliability and complicate error handling and optimization processes.
Specialized Expertise	Quantum programming models often require specific expertise, making it inaccessible for those without a specialized background. The lack of high-level abstractions force the developers to deal with low-level operations.

across multiple platforms. They demonstrated with implementations using Qiskit and Cirq.

The unreliability of NISQ computers by proposing Quantum DevOps, which integrates development, testing, and operations of quantum algorithms, is addressed by [60]. This proposal involves regularly evaluating quantum instances and applying continuous integration and delivery processes to ensure reliable quantum computations.

The integration of quantum computing into classical information systems using web engineering is presented by [61]. They propose an orchestration method and lifecycle architecture for quantum web services, utilizing DevOps techniques to manage the complexity and ensure efficient development and production of hybrid systems.

The deployment challenges of quantum software applications in production environments are addressed by [62]. They propose a hybrid classical–quantum cloud architecture blueprint, demonstrated with a quantum-based auction matching algorithm for P2P energy trading. It emphasizes continuous integration and deployment to optimize quantum resources and performance.

The current environments require programmers to manage low-level hardware features, hindering broader adoption [63]. The authors propose an abstraction hierarchy inspired by classical computing models, separating programming, execution, and hardware concerns. This hierarchy improves the organization and understanding of the software stack and is demonstrated through the eigenvalue estimation problem, providing insights into error mitigation and correction. The results show that this approach can facilitate more productive and scalable quantum programming, emphasizing the need for further refinement and broader application of these models.

All the reviewed proposals present several limitations, including latency, scalability, standardization, and interoperability issues. We summarize these limitations in Table 1. As all these papers highlight, integrating of QC with classical systems presents numerous challenges and opportunities, requiring innovative approaches and methodologies. Existing research has focused on various aspects of this integration, from addressing the technical complexities of hybrid quantum applications to proposing new software engineering techniques and hybrid life cycles. In summary, all the frameworks share the common weaknesses of complexity in hybrid quantum–classical integration, scalability challenges, performance and latency concerns, a lack of standardization, unreliable quantum hardware, and the need for specialized expertise. These detected weaknesses are consistent with the results and findings presented in [64].

Our proposal introduces an SPL approach for managing the complexity of hybrid quantum–classical software development. This method

aligns with Model-Driven Engineering principles, aiming to handle high variability and facilitate the reuse of software components. Similar to the works reviewed, our approach emphasizes the integration of quantum and classical elements, focusing on modularity and reuse. However, it distinguishes itself by leveraging SPL principles to manage variability systematically, offering a structured framework that supports the development of diverse software products through feature modeling. Our proposed SPL approach builds on QSE foundations, aiming to provide a robust framework for managing variability and enhancing the development efficiency of hybrid quantum–classical software. The subsequent sections of this paper will delve deeper into the SPL methodology, detailing its implementation and potential to streamline the development of complex quantum–classical systems.

4. Proposal

As QC becomes a transformative force, adapting and enhancing classical software with quantum functionalities is increasingly essential. This research introduces an innovative feature model designed to facilitate this integration, providing a new perspective on software development in the quantum era. Therefore, the main proposal of this research is the application of an SPL approach for the development of hybrid software systems. The core of this proposal is an FM specifically modeled to support hybrid software product lines. It should be noted that this proposal is not aimed at adapting SPL techniques for quantum software but its application to the challenging scenario of hybrid software systems.

Firstly, Section 4.1 explains the difference with SPL in classical software and why quantum software is a differential factor. Section 4.2 explains the overall process and how the proposed FM can be used. Finally, Section 4.3 presents the FM and explains how this model can express variability in quantum–classical software.

4.1. Rationale of this proposal

The difference between using the SPL approach in classical software systems and its application in hybrid systems that integrate quantum computing lies in several key aspects. QC introduces significant differential factors that justify its specialized consideration within the SPL paradigm.

- **Nature of Quantum Computing.** Quantum computing is based on principles of quantum mechanics, such as superposition and entanglement, which have no equivalents in classical computing.

This difference means that developing quantum software requires knowledge and management of concepts radically different from those used in classical computing.

- **Variability and Configurability.** In classical SPL, variability is mainly managed within the context of software functionalities and configurations. In the quantum context, variability must also consider specific aspects of quantum algorithms, quantum error management, and the suitability of the software for different types of quantum hardware, which can vary significantly in their capabilities and how they perform quantum operations [65].
- **Development and Validation.** The tools, languages, and development environments for quantum computing are in the early stages of evolution compared to the mature ecosystems of classical development [66]. This stage of evolution adds a layer of complexity to managing product lines as new tools and processes adapted to quantum computing must be incorporated. Additionally, the validation of quantum software requires special methods and techniques due to the probabilistic nature of quantum results [67]. This validation differs from the deterministic testing techniques used in classical computing and necessitates adapting the SPL principles and foundations to handle these challenges adequately.
- **Interaction Between Quantum and Classical Components.** Integrating quantum and classical components in the same hybrid system introduces additional challenges in terms of communication and synchronization between these components [8]. This challenge necessitates new strategies within the SPL to manage and optimize the interaction between parts of the system operating under fundamentally different computational paradigms.
- **Rapid Evolution of the Quantum Field.** Innovation Pace: Quantum computing is a rapidly evolving field, with constant advances in hardware and algorithms [6]. This advance requires the SPL approach to be highly adaptable and capable of quickly incorporating new advancements and paradigms into its product lines. This requirement is less pronounced in classical computing.

The genuine characteristics of quantum software make it essential to develop an SPL approach that supports these capabilities and is flexible and robust enough to adapt to the specific nature and requirements of quantum software.

In summary, integrating quantum computing within the SPL approach introduces unique challenges and opportunities that justify a differentiated treatment. This specialized approach optimizes the development of hybrid systems and allows for the full potential of quantum computing to be realized in conjunction with classical systems.

4.2. Application process of hybrid software product lines

In line with the MDE principles, the process for using this proposal involves several kinds of models at different abstraction levels, as well as different roles that operate at different levels and use specific models. Fig. 3 summarizes the process for applying the proposal, which considers three different abstraction levels.

4.2.1. Computing-independent model

This level supposes the highest abstraction and considers specific domain details without paying attention to computing details. These details are modeled through the Domain FM (FM-D). The proposed FM for hybrid software systems (FM-H), which is at an intermediate level, can be applied to different domains through the FM-D. At the high abstraction level, the central role involved is that of the domain expert, who can use existing FM-Ds, or define their own. Many existing FM-Ds exist in various domains or industrial sectors (e.g., [68–72]). FM-D is computing-independent and only focused on defining the expected software features that a software product line in a particular domain might/should consider. Thus, the domain expert's work at this

level will be to select the eligible expected features that the possible software variants could have to support through the SPL. This selection is defined in the so-called application configuration model (CM-D). This configuration model could be tailored with specific constraints between different features in different configurations (see CC - Constraints Configuration- in Fig. 3).

In this layer, the application designer plays a crucial role in bridging the gap between domain knowledge and software design. While domain experts define the high-level features and constraints in the FM-D, the application designer is responsible for translating these into a coherent configuration model that can guide the development process. They use their expertise in both the domain and software engineering to select relevant features from the FM-D and refine them into a CM-D that aligns with the goals and constraints of the specific application. The application designer ensures that the selected features are feasible for implementation and adaptable to the target environment, contributing significantly to the initial shaping of the product line.

4.2.2. Platform-independent model

The second level is not computing-independent anymore. However, it is still platform-independent (see Fig. 3), i.e., specific software development details are abstracted at this level. The application designer, who should know about the domain, is the primary role at this level. Application designers will use our proposed FM for hybrid software systems (FM-H). Similarly to the previous level, they will select the features of developing hybrid software. This feature subset is integrated into the hybrid software configuration model (CM-H) and are equally constrained in both levels. First, regarding some quantum computing aspects (e.g., “if the feature ‘gate-based’ is selected, then ‘quantum-Lang’ should be ‘Qiskit’”). Second, the CC model could include constraints between two levels (e.g., “if ‘feature 1’ is selected, then it could be supported by ‘quantum’, including both features, ‘gate-based’ or ‘annealing’ features”). This option indicates that ‘feature 1’ can only be supported by quantum software.

4.2.3. Platform-specific model

Finally, this level considers specific development details to produce software products (see Fig. 3). Here, the leading roles are quantum and classical software developers to produce software variants by considering CM-D and CM-H alongside constraints defined in CC. At this point, developers can use the existing chain of tool sets in the SPL field to produce and evolve software variants. Of course, the tool-set should be extended adapted and adapted to cover details of quantum software development.

In the platform-specific layer, the application designer continues to play a vital role by guiding the software developers in implementing the selected features and ensuring they align with the overall configuration model. The application designer collaborates with quantum and classical software developers to translate the configuration into concrete software components. They also ensure that the developers understand the dependencies and constraints specified in the configuration model, facilitating the creation of a cohesive software product. This role involves continuous oversight to address any ambiguities or conflicts that may arise during implementation, ensuring that the final product accurately reflects the initial design decisions.

4.3. Feature model for hybrid software

FMs has traditionally used in SPL to represent and manage the variations and commonalities in product families [73–75]. Then, it is crucial to consider how quantum features can be modeled, configured and adapted when SPL is applied to hybrid software systems. Managing variability in the context of quantum software using FM involves modeling, configuring, and adapting quantum-related features, considering quantum software relationships, constraints, and particularities. This integration promises to develop more modular, reusable, and adaptable

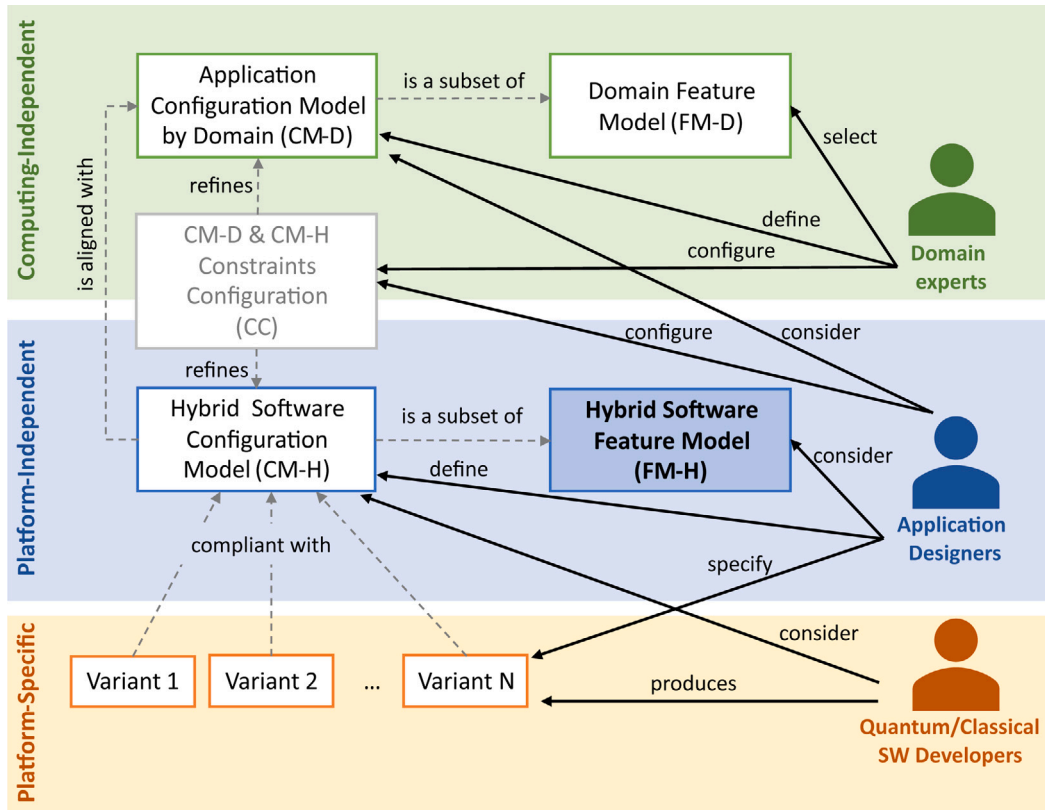


Fig. 3. Process overview for the proposed FM for hybrid software.

quantum software but also presents unique challenges that must be considered. An initial discussion follows concerning the main sections, points of variation, and identified features.

The approach can be defined following a Feature Modeling perspective in which the SPL approach combines MDE and Requirement Engineering principles. This perspective attempts to use SPL/MDE to manage variability technology independently, as it was explained in Section 4.2.

According to the different abstraction levels, in a hybrid software system, the variability presents an initial set of features related to the functionality, e.g., the set of quantum algorithms, the programming languages, the integration mechanisms, and the hardware, were considered. Then, we reviewed each feature in detail, and a subset of features and relationships between them were identified. The details of this model are presented in Fig. 4.

FMs provide various modeling concepts, such as optional and mandatory features, a feature hierarchy, feature groups, and cross-tree constraints, as illustrated in Fig. 4. Our proposal, called *Hybrid Quantum-Classical SPL (HQC-SPL)*, exhibits five main features, two of which are required (functionality and programming languages). The other three features are optional (quantum algorithms, integration mechanisms, and hardware-based constraints). The remaining child features are related to their parent feature by an OR relation. The only exception is the case of the HW-based-constraint feature, in which the child features represent one of the three possible levels of hardware constraint over a possible software implementation (basic functionality, quality, and error correction) [76].

The *functionality* feature represents a hybrid system, considering a software system's quantum and classical elements. The feature *quantumAlgorithm* describes the diverse options as regards implementing the quantum algorithms in a particular solution. The feature *programmingLang*, includes the possibilities of classical or quantum programming languages. The integration mechanisms consider the feature *integrationMech*, and the child features *micro-Ss* or *quantumSOA* represent a

potential integration using micro-services or quantum SOA services [77, 78], respectively.

In addition to the relationships between features, cross-tree constraints are allowed. These constraints consider two cases: *« requires »* and *« excludes »*. The *« requires »* relationship between *feature A* and *feature B* means that the selection of *feature A* implies the selection of *feature B*. On the other hand, the *« excludes »* relationships between *feature A* and *feature B* means that *feature A* and *feature B* cannot be part of the same product. For example, in our proposal, selecting the feature *quantum* implies that the *HW-based constraint* must be selected. Another case could be selecting the feature *quantumAlgorithm*, which implies that the *quantumLang* must be selected.

5. Example of application

We propose to illustrate the proposal's usage by providing a running example consisting of a new or evolved software system for a global supply chain logistics company. Section 5.1 explains the target hybrid software application to be developed following the proposed SPL approach, while Section 5.2 details the configuration model for the proposed logistic domain. Section 5.3 presents how the proposed FM for hybrid software systems is used in combination with the domain configuration model. Finally, Section 5.4 illustrates how a specific software product is produced with the proposed SPL solution.

5.1. Target hybrid software application

Logistics companies are constantly looking to optimize transportation routes to reduce costs and delivery times. QC can provide faster and more accurate solutions to complex optimization problems.

A hybrid system could be developed in which the user interface, data management, and other functions are handled in a classical system while optimization calculations are performed in a quantum system.

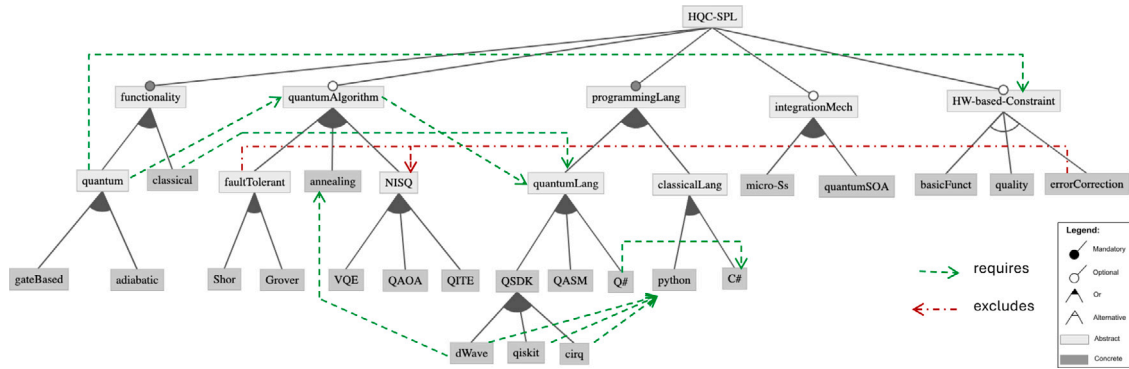


Fig. 4. FM for the HQC-SPL proposal.

Variability features could include different quantum optimization algorithms, adaptability to different quantum hardware, and quantum simulation options.

We need an SPL approach since several local offices will be operating different software versions. Those variants could be due to, e.g., the retail they manage, the available distribution mode (air, sea, truck, or rail), or the management activities these offices should be accountable for (optimizing routes, last-mile delivery, etc.). Together with domain-specific variability, the software is hybrid; therefore, quantum software could support some functionality. There are already proposals in this regard [79,80].

To provide a simplified example, the main functionality of the system consists of the following four functional requirements.

- RF1. Effective storage and inventory management
- RF2. Computing optimal shipping routes
- RF3. Monitoring last-mile delivery
- RF4. Forecasting the expected demand

RF1 and RF3 are more closely related to the daily operative level and are prone to be supported by classical software. RF2 and RF4 are, meanwhile, more closely related to the managerial/tactical level. The nature of these two functionalities (optimization and estimation) has sufficient potential to be supported by quantum software.

5.2. Domain configuration model

According to the process presented in Fig. 3, the first step is to define the application configuration model (CM-D) by considering a general domain feature model (FM-D) for the logistics domain. Fig. 5 presents the proposed configuration model for this domain and this example. This model is based on the ontology provided by [81] and is inspired by the SPL approach to logistics proposed by [82]. The proposed configuration model is an example subset of features for this domain for the specific logistics company presented in this example. The whole FM for the logistics domain could be larger and more precise than this and could be configured differently for another logistics company.

The configuration model presented (see Fig. 5, left-hand side) has two significant branches named 'Process' and 'Resource', which deal with specific features to deal with the ordinary, core activities of the logistics company. For example, these branches include material flows, different locations, which could affect software features, kinds of transportation, among others. The branch denominated as 'Management' (see Fig. 5, right-hand side) is devoted to the definition of the four main functional requirements:

- RF1 - 'StorageMgmt'
- RF2 - 'DefineRoutes'
- RF3 - 'LastMileTracking'
- RF4 - 'ForecastDemand'

5.3. Hybrid software configuration model

Having defined the system variability concerning a specific domain, in this example, i.e., the logistics domain, the proposed feature model for hybrid software system can be used. In this case, the model could be used individually for each main functionality (RF1 to RF4), or it could be added as a full branch in the root configuration model (CM-D) previously defined. In this example, we propose to use it as an additional branch, which is then configured using constraints between features. The whole configuration model is not graphically represented together with the CM-D model presented in Fig. 5 for readability reasons. Instead, the hybrid software configuration model (CM-H) is presented in Fig. 6. Together with this, Table 2 provides some of the constraints between elements of that branch (CM-H) and the whole configuration model (CM-D).

In this example, we suppose that RF1 (the storage management) and RF3 (last-mile tracking) will be supported in all software variants by means of classical software, which will be specifically coded in Python. This is modeled by means of the first three constraints.

Then, RF2 (defining routes) will, therefore, be supported by a quantum annealing algorithm with which to determine optimal routes like that proposed by Ariño and Palacios [83]. This approach could be difficult to be scaled for areas with thousands of nodes and possible edges. Hence, this optimization approach sometimes is reduced to smaller problems by previously applying a clustering algorithm based on classical software [84], which could extract smaller graphs by customer, area, etc. C4 to C6 consequently defines a mandatory constraint with which to define the fact that the route optimization feature should always be supported by adiabatic quantum computing, with an annealing algorithm, which should be coded in D-Wave with a clear dependency on Python (C7). Alternatively, some classical software could be used for the clustering pre-processing. This fact does not require any constraints, since the FM proposed in Fig. 6 can be used to produce software variants with this feature. With regard to the integration between classical and quantum software, this product line forces the use of microservices when needed. There is no need to define a constraint for it, since the configuration model already forces the integration of microservices into the entire product line (see Fig. 6).

Finally, RF4 (forecasting expected demand) will be supported by a Quantum Approximate Optimization Algorithm (QAOA) [85], which performs very well with scheduling and optimization problems in the logistics domain [86]. Of course, this functionality could be partially supported by classical software, which plays the role of the request driver and the QAOA solution interpreter. The configuration model presented in Fig. 6 already supports this feature. Domain-specific constraints are not defined here owing to space limitations. However, this approach could serve to define, for example, specific software product configuration depending on the location. For instance, the offices in the USA should use products supporting quantum software to optimize routes based on quantum annealing algorithms. However, offices in the EU support this feature by employing classical software. These specific constraints are omitted from Table 2.

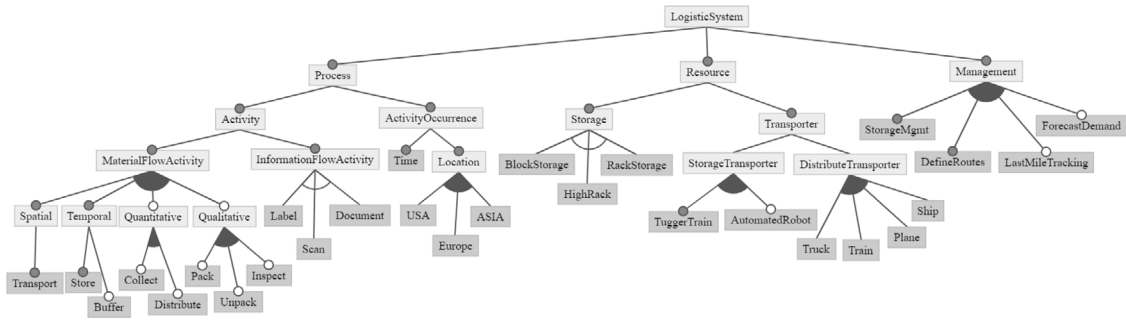


Fig. 5. FM for the HQC-SPL proposal.

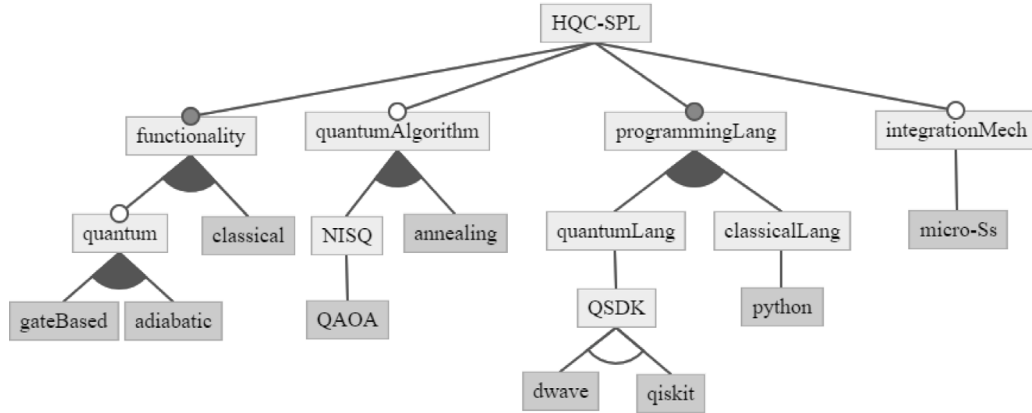


Fig. 6. Configuration model for the logistic hybrid software system.

Table 2
Identified constraints among features for the example hybrid SPL.

Id	From	Constraint	To
C1	StorageMgmt	requires	classical
C2	LastMileTracking	requires	classical
C3	classical	requires	python
C4	DefineRoutes	requires	adiabatic
C5	adiabatic	requires	annealing
C6	adiabatic	requires	dwave
C7	dwave	requires	python
C8	ForecastDemand	requires	QAOA
C9	QAOA	requires	gateBased
C10	QAOA	requires	qiskit
C11	qiskit	requires	python

5.4. Example of software product instance

In this section, we present a final example of a particular product variation (see Fig. 7) produced with the SPL approach that considers the models above in the proposed example.

In an SPL-based hybrid quantum–classical system, low-level features and constraints, such as hardware limitations and platform-specific requirements, significantly influence the design and coding of each product. Key constraints include quantum hardware limitations (e.g., qubit count, error rates), classical hardware performance, and platform dependencies (e.g., D-Wave Ocean SDK, Qiskit). However, such variability concerns are precisely managed by the instantiated FM. To build a Minimum Viable Product (MVP), tools such as Docker and Kubernetes can integrate classical components (e.g., inventory management) with quantum algorithms (e.g., D-Wave quantum annealing for route optimization). Some existing SPL tools like FeatureIDE and Pure::Variants manage variability and automate product configuration.

Concerning the domain variabilities, let us imagine the software product for a European office that operates only in Europe and manages only two main material flow activities: transport and storage. For the specific software variant, only the tugger train (as a mandatory feature) and the truck management feature are considered, since this office manages only a truck fleet as distribution transportation. In this office, the main management functionalities are RF1 and RF2 (i.e., storage management and the definition of optimal routes).

With regard to the features of the hybrid software systems, and according to the constraints previously defined, this variant considers a quantum annealing algorithm implemented in D-Wave for the optimization of truck routes (RF2). Additionally, classical software coded in Python will be used to support the storage management (RF1), along with the integration between classical and quantum software through the use of micro-services.

The resulting product is a direct instantiation of the specific FM depicted in Fig. 7. Thus, a minimum viable product (MVP) can be built directly by reusing software artifacts and components in a common software repository for the whole SPL. Fig. 8 presents the outgoing product in this application example, which is abstracted through a UML class diagram. It should be noticed that this UML model hides many others implementation details that are not relevant to illustrate this running example. Some design patterns like Model/View/Controller (MVC) have been applied in the model presented in Fig. 8. Apart from the automatic reuse of some common software artifacts, specific design patterns could be systematically applied on the instantiation of a product variant from the SPL.

With regard to the elements included in the UML diagram (see Fig. 8), classes like *Product*, *Inventory*, *Storage* come from the Storage Management feature, which is a functionality supported by classical software. On the other hand, *OptimizationProblem*, *Route*, *TuggerTrain* and *Order* classes are from the Route Optimization feature.

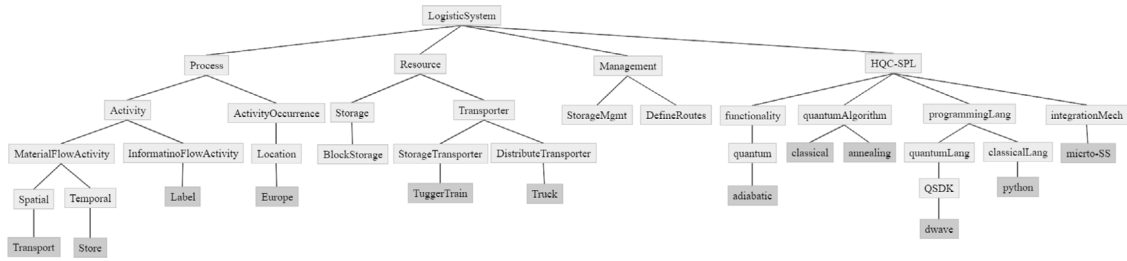


Fig. 7. An example of software product variant for the logistic hybrid software system.

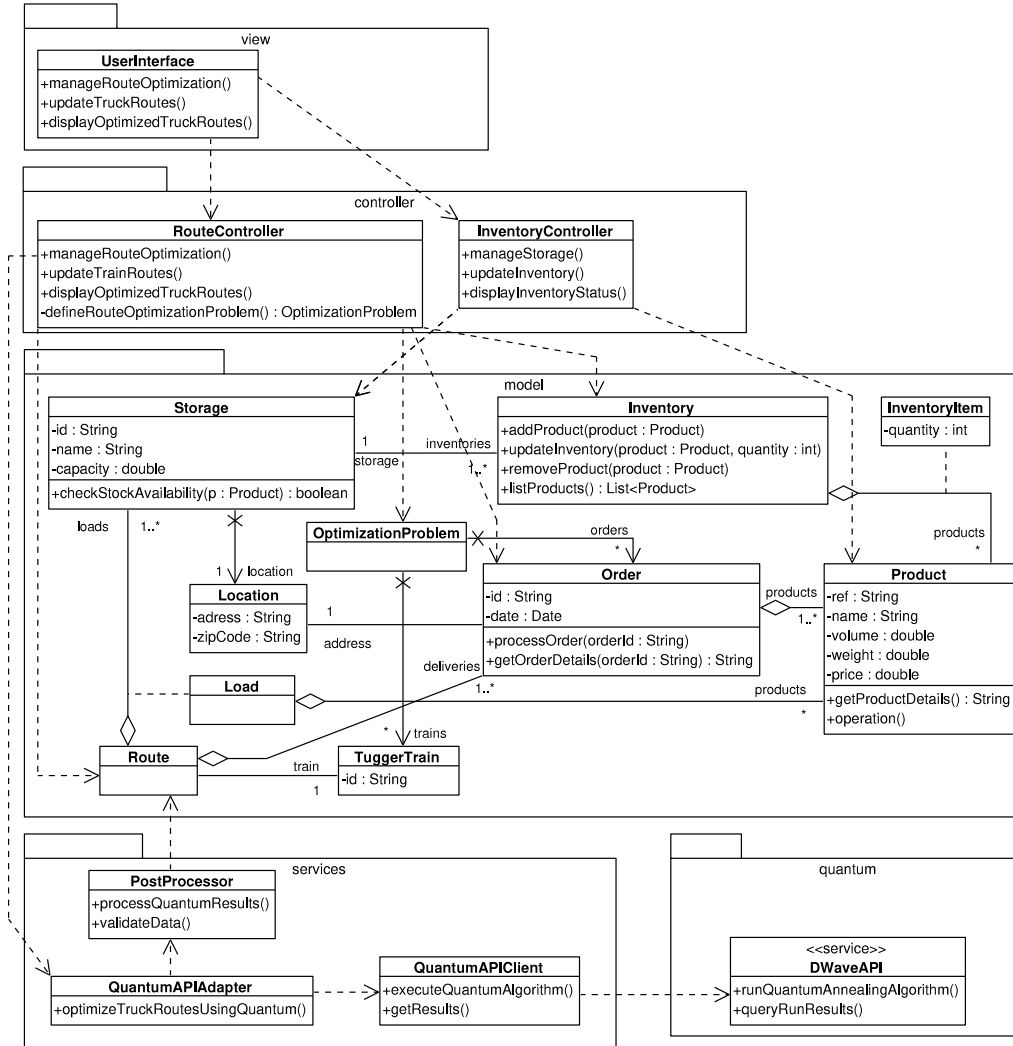


Fig. 8. The outgoing variant as a minimum viable product represented as an UML model.

The Route Optimization feature leverages quantum computing, specifically quantum annealing using the D-Wave API. The *RouteController* class interacts with both classical and quantum components. It uses classical software for initial data processing (for example, preprocessing routing data) and then sends the optimization problem to the quantum processor via a *QuantumAPIClient* that communicates with the D-Wave API. Before this, the *QuantumAPIAdapter* converts data from the classical format to a format that can be processed by the D-Wave quantum annealer. The quantum algorithm then runs, optimizing the routes. After receiving the quantum solution, the system may apply classical post-processing to refine or interpret the quantum results. This ensures that the final route plan is practical and ready for execution.

Using SPL, this architecture allows the same core inventory management components to be reused in other variants of the logistics system, even if different quantum or classical algorithms are selected in those variants. For example, another variant could use a classical route optimization algorithm, but the inventory management system would remain the same, ensuring maximum reuse of components. In SPL, software reuse is a fundamental principle that ensures efficient product configuration and generation across different variants.

6. Discussion

This section provides an in-depth exploration of the multifaceted implications and insights derived from our proposed framework with

which to develop hybrid quantum–classical software using an SPL approach. The discussion is structured in such a way as to provide a comprehensive understanding of the benefits and challenges associated with this innovative proposal. Section 6.1 explores the overarching purpose and potential gains of our proposal, highlighting how it paves the way for groundbreaking advances in the development of hybrid software systems. Section 6.2 then presents the lessons learned for researchers and practitioners, highlighting the critical insights and practical strategies of our research. Finally, in Section 6.3, we address the limitations inherent to our current approach, acknowledging the constraints and areas that require further refinement.

6.1. Purpose and gains of the proposal

The proposal introduces an innovative method with which to manage the complexity caused by the combination of quantum and classical parts in hybrid systems, improving their construction and reuse so as to pave the way for advanced solutions to complex computational problems. It emphasizes enhancing development efficiency and adaptability by applying SPLs principles. This approach is poised to revolutionize the field by seamlessly blending the capabilities of classical and quantum software.

Our proposal tackles several critical complexities in hybrid quantum–classical systems, improving automation, quality, reuse, and the lack of skills for quantum development. Next, we detailed every one of these complexities.

6.1.1. Improving automation

Due to the differences in their computational models, hybrid systems pose important challenges. Our approach automates the generation of hybrid code by leveraging high-level abstractions, simplifying the orchestration between quantum and classical elements, and reducing manual coding effort.

6.1.2. Quality of hybrid systems

Ensuring the quality of hybrid systems is complex, mainly due to quantum noise, decoherence, and the probabilistic nature of quantum systems. Our solution incorporates quantum error correction techniques and automated verification tools to enhance the robustness and reliability of these systems.

Software testing is another important task when a product is delivered from the SPL. The proposed approach can contribute in several ways.

- **Commonality and Variability Testing:** SPL allows the testing of common components (e.g., core classes like Inventory, Product, Order in the example of application) across all variants. This minimizes the need for repeated testing for each variant, focusing only on the specific differences in features.
- **Automated Testing for Variants:** SPL frameworks often come with tools to generate test cases automatically for each product variant. Given that the core components remain the same, testing efforts can be concentrated on testing the different feature combinations (e.g., classical vs. quantum optimization in route planning).
- **Regression Testing:** Since the FM defines the variability, it is easier to perform regression testing on reused components whenever changes occur. For example, if a bug is found in the Product class, it can be fixed once, and the fix will propagate across all variants that reuse this class.

6.1.3. Increasing reuse

The lack of reusable components in QC slows development. We address this by providing a modular framework with reusable quantum and classical components, enabling efficient reuse across different applications and reducing development time.

Our proposal reveals that the principles of SPLs can be adapted to the domain of QSE, facilitating the identification and management of the variability present in quantum and classical computing paradigms. We applied SPL concepts to develop a framework that supports the variability inherent in QC technologies and programming languages, which can additionally accommodate the specific constraints of quantum algorithms.

The proposed framework marks a significant step towards bridging the gap between quantum and classical software. The adaptability of the framework allows for the reuse of diverse software components that supports the accommodation of a wide range of quantum computing technologies, which is crucial given the rapid evolution of this field.

6.1.4. Mitigating the lack of skills in quantum development

Quantum development requires quantum physics and mathematics expertise, creating a steep learning curve. Our proposal lowers this barrier by offering high-level tools and automated frameworks that allow developers with classical programming skills to contribute to quantum projects, promoting broader participation in quantum development.

Finally, we propose simplifying the development of hybrid systems, improving automation, system quality, and code reuse, and addressing the skill gap in quantum programming, thereby accelerating the advancement of QC technologies.

6.2. Lessons learned

During the development of this work and its comparison with experts in the field, we have learned several lessons:

- **Importance of Interdisciplinary Collaboration** The research highlights the need for collaboration between SE and QC.
- **Challenges in Integrating Methodologies** Insights into the unique challenges of integrating classical methodologies with quantum artifacts have been gained.
- **Potential of SPL in New Domains** The application of SPL in the quantum context opens up new areas of research in variability management and software customization. Modularity and reuse are emphasized in order to improve efficiency and scalability when developing hybrid software.
- **For researchers** The proposal underscores the importance of interdisciplinary collaboration between SE and QC, highlighting the unique challenges involved in adapting SE classical methodologies to quantum and hybrid software systems.
- **For practitioners** The proposal offers practitioners a practical framework in which to address the complexity inherent in developing hybrid quantum–classical software. It emphasizes the benefits of modularity and reuse, along with critical strategies with which to enhance efficiency and scalability when developing hybrid software systems. Moreover, the approach makes it possible to manage the inherent complexity of hybrid quantum–classical software.
- **Quantum ecosystem approach** This approach could also be used in combination with another approach such as Quantum DevOps [60]. Our approach could be useful for managing variability and how it affects certain operation activities, e.g., selecting the most appropriate quantum device, or recognizing how to test, optimize and execute a specific quantum algorithm.

6.3. Limitations

A notable limitation of the proposal is its potential reliance on specific tools and platforms, which may impede its adaptability and applicability in various technological contexts. The effectiveness and efficiency of the proposed framework could be significantly influenced by the tools and platforms used, posing a challenge to its widespread adoption and implementation.

Implementing the proposal requires technical QC expertise, which is not yet commonly available. This lack of expertise is a considerable barrier to the development of quantum–classical software, potentially decelerating the adoption and refinement of the proposed methodologies and principles. In addition to that, effective collaboration between domain experts and application designers is essential for the success of this approach. Domain experts provide the foundational knowledge about the domain, identifying essential features, commonalities, and variability. Application designers then use this information to create a viable configuration model that can guide the software development process. To manage this collaboration, regular communication and iterative review processes are crucial. Joint workshops and design sessions can facilitate a shared understanding of domain requirements and technical constraints. Furthermore, establishing clear documentation standards and decision-making protocols can ensure that domain knowledge is accurately captured and translated into configuration models, fostering a seamless transition from domain analysis to software implementation. This collaboration ensures that the final product not only meets the technical requirements, but also aligns closely with the domain-specific needs.

Another limitations refers to the conflicts that may arise during development over technical or project decisions. According to [87], applying the decision management process allows teams to objectively analyze alternatives, weigh them against established criteria, and select the most valuable solution. This systematic and structured approach ensures that conflicts are settled through data-driven decision-making, enhancing collaboration and aligning the team on the best pathway ahead.

In addition, a great opportunity for improvement is the second generation of SPLs [88,89], which considers both software and hardware issues to integrate multiple systems. Actually, in the current landscape of quantum computing, quantum software co-design with hardware is a common practice due to the state of maturity of quantum software technology [65,90,91]. While there are ongoing efforts to abstract quantum hardware details (e.g., Qiskit, Cirq), these cannot be fully ignored due to unique constraints like qubit coherence and gate fidelity. Our proposal already considers some hardware-related aspects, such as differentiating between gate-based and annealing quantum computing. However, integrating the second generation SPLs would allow for a more refined feature modeling that explicitly manages dependencies between hardware capabilities and software features, leading to better system integration. Although this deeper integration is outside the current scope, it represents a valuable direction for future research.

A related limitation is about the potential usage of Dynamic SPLs (DSPLs). While our proposal focuses on traditional SPL approaches for managing variability in hybrid software systems, we acknowledge that DSPLs offer significant potential benefits in this domain. DSPLs can handle runtime variability, adapting to the evolving nature of quantum hardware and algorithms. They can dynamically reconfigure quantum and classical resources, optimize workload distribution, and manage fault tolerance. However, integrating DSPL into hybrid quantum systems involves complex runtime decision-making and remains an open research question in both SPL and quantum software development. Therefore, a comprehensive exploration of DSPL applications in this context is outside the scope of this paper.

7. Conclusions

The growing diffusion of quantum computing is making true quantum software engineering more and more important, allowing the extension and adaptation of existing good practices in traditional software engineering.

This study presents a preliminary application of SPL principles to the domain of QC in order to manage the variability in hybrid (classical/quantum) software systems. The proposed approach successfully employs feature modeling to represent and handle this variability,

allowing a clear delineation of commonalities and differences across a spectrum of software products. This structured approach to variability management is crucial as regards addressing the unique challenges of integrating quantum algorithms into classical computing environments. The use of feature models not only facilitates the customization and scalability of hybrid systems but also provides a systematic way in which to capture the evolving landscape of quantum computing capabilities. The conclusions underscore the importance of robust variability management in realizing the full potential of quantum–classical software development methods and set the stage for future research that will expand upon this initial approach and FMs to accommodate the rapid evolution of quantum software technologies.

Our research agenda will focus on expanding the scope of the SPL framework so as to encompass a broader range of quantum computing scenarios and to refine the granularity of feature models. We aim to develop tool support with which to automate the configuration and deployment of hybrid software systems, addressing the dynamic nature of quantum software development. In future work, we aim to explore the integration of Dynamic SPLs and second-generation SPLs to further enhance the adaptability and robustness of hybrid quantum–classical software systems, addressing runtime variability and the interplay between software and hardware components.

We additionally plan to carry out empirical studies and industry collaborations are planned to validate the framework in practical settings, thus ensuring that it meets the needs of both developers and stakeholders in the quantum computing ecosystem. Thus, the future direction of this proposal involves the development of specific tools and frameworks with which to enhance the application of SPL in hybrid software systems. There is also a focus on conducting empirical validations and case studies so as to assess the efficacy of the proposal in various scenarios.

Furthermore, the proposal aims to explore new application areas, and we plan to investigate how it can be adapted and applied in diverse domains such as cryptography, process optimization, and complex system simulation. This approach underscores the commitment to expanding the scope and impact of the identification of the variability in quantum and classical software.

CRedit authorship contribution statement

Samuel Sepúlveda: Writing – review & editing, Writing – original draft, Validation, Investigation, Formal analysis, Conceptualization. **Ricardo Pérez-Castillo:** Writing – review & editing, Writing – original draft, Validation, Investigation, Formal analysis, Data curation, Conceptualization. **Mario Piattini:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Samuel Sepúlveda is supported by Grant ANID, Chile, FONDECYT DE INICIACIÓN EN INVESTIGACIÓN, Project No 11240702.

Pérez and Piattini thank the PID2022-137944NB-I00 (SMOOTH Project) and PDC2022-133051-I00 (QU-ASAP Project) funded by MCIN/AEI/10.13039/501100011033 and by the “European Union NextGenerationEU/PRTR”, and the Financial support for the execution of applied research projects, within the framework of the UCLM Own Research Plan, co-financed at 85% by the European Regional Development Fund (FEDER) UNION (2022-GRIN-34110).

Data availability

No data was used for the research described in the article.

References

- [1] M. Felderer, D. Taibi, F. Palomba, M. Epping, M. Lochau, B. Weder, Software engineering challenges for quantum computing: Report from the first working seminar on quantum software engineering (WSQE 22), *ACM SIGSOFT Softw. Eng. Notes* 48 (2) (2023) 29–32.
- [2] A. Carleton, E. Harper, J. Robert, M. Klein, D. De Niz, E. Desautels, J. Goodenough, C. Holland, I. Ozkaya, D. Schmidt, F. Shull, J. Herbsleb, J. Foreman, R. Cunningham, Architecting the Future of Software Engineering: a National Agenda for Software Engineering Research and Development, Tech. Rep., Softw. Eng. Inst., Pittsburgh, PA, 2021.
- [3] L. Barbosa, Software engineering for ‘quantum advantage’, in: *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, Association for Computing Machinery, IEEE, New York, United States, 2020, pp. 427–429.
- [4] J. Zhao, Quantum software engineering: Landscapes and horizons, 1 (1), 2020, pp. 1–34, [arXiv preprint arXiv:2007.07047](https://arxiv.org/abs/2007.07047).
- [5] Y. Alexeev, A. McCaskey, W. De Jong, Introduction to the special issue on software tools for quantum computing: Part 1, *ACM Trans. Quantum Comput.* 3 (3) (2022) 1–3.
- [6] M.A. Serrano, R. Pérez-Castillo, M. Piattini, *Quantum Software Engineering*, Springer, Switzerland, 2022, <https://doi.org/10.1007/978-3-031-05324-5>, URL <https://link.springer.com/book/10.1007/978-3-031-05324-5>.
- [7] M. De Stefano, F. Pecorelli, D. Di Nucci, F. Palomba, A. De Lucia, Software engineering for quantum programming: How far are we? *J. Syst. Softw.* 190 (2022) 1–15.
- [8] F. Leymann, J. Barzen, Hybrid quantum applications need two orchestrations in superposition: A software architecture perspective, 2021, [arXiv:2103.04320](https://arxiv.org/abs/2103.04320).
- [9] B. Weder, J. Barzen, F. Leymann, D. Vietz, *Quantum Software Development Lifecycle*, Springer International Publishing, Cham, 2022, pp. 61–83, [http://dx.doi.org/10.1007/978-3-031-05324-5_4](https://doi.org/10.1007/978-3-031-05324-5_4).
- [10] R. Pérez-Castillo, M.A. Serrano, J.A. Cruz-Lemus, M. Piattini, Guidelines to use the incremental commitment spiral model for developing quantum-classical systems, *Quantum Inf. Process.* 24 (1&2) (2024) 0071–0088.
- [11] P.C. Clements, L. Northrop, *Software Product Lines: Practices and Patterns*, Addison-Wesley, NJ, USA, 2001.
- [12] J. Bosch, G. Florijn, D. Greefhorst, J. Kuusela, J.H. Obbink, K. Pohl, Variability issues in software product lines, in: *International Workshop on Software Product-Family Engineering*, Springer, Berlin, Heidelberg, Bilbao, Spain, 2001, pp. 13–21.
- [13] M.A. Babar, L. Chen, F. Shull, Managing variability in software product lines, *IEEE Softw.* 27 (3) (2010) 89–94.
- [14] M. Galster, D. Weyns, D. Tofan, B. Michalik, P. Avgeriou, Variability in software systems—a systematic literature review, *IEEE Trans. Softw. Eng.* 40 (3) (2014) 282–306.
- [15] K. Czarnecki, A. Wasowski, Feature diagrams and logics: There and back again, in: *11th International Software Product Line Conference, SPLC 2007*, IEEE, Kyoto, Japan, 2007, pp. 23–34.
- [16] H. Arboleda, J.C. Royer, *Model-Driven and Software Product Line Engineering*, ISTE Limited, London, England, 2012.
- [17] M. Sinnema, S. Deelstra, Industrial validation of COVAMOF, *J. Syst. Softw.* 81 (4) (2008) 584–600.
- [18] K. Pohl, G. Böckle, F.J. van Der Linden, *Software Product Line Engineering: Foundations, Principles and Techniques*, Springer Science & Business Media, Berlin, Germany, 2005.
- [19] S. Apel, D. Batory, C. Kästner, G. Saake, Software product lines, in: *Feature-Oriented Software Product Lines: Concepts and Implementation*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 3–15, [http://dx.doi.org/10.1007/978-3-642-37521-7_1](https://doi.org/10.1007/978-3-642-37521-7_1).
- [20] J.-M. Horcas, M. Pinto, L. Fuentes, Software product line engineering: A practical experience, in: *Proceedings of the 23rd International Systems and Software Product Line Conference - Volume a, SPLC '19*, Association for Computing Machinery, New York, NY, USA, 2019, pp. 164–176, [http://dx.doi.org/10.1145/3336294.3336304](https://doi.org/10.1145/3336294.3336304).
- [21] C.W. Krueger, P.C. Clements, Second generation systems and software product line engineering, in: *Proceedings of the 17th International Software Product Line Conference*, 2013, pp. 283–284.
- [22] J. Krüger, T. Berger, An empirical analysis of the costs of clone- and platform-oriented software reuse, in: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in: *ESEC/FSE 2020*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 432–444, [http://dx.doi.org/10.1145/3368089.3409684](https://doi.org/10.1145/3368089.3409684).
- [23] S. Apel, D. Batory, C. Kästner, G. Saake, *Feature-Oriented Software Product Lines*, Springer-Verlag Berlin An, Berlin, Germany, 2016.
- [24] F. Ahmed, L.F. Capretz, Best practices of RUP® in software product line development, in: *2008 International Conference on Computer and Communication Engineering*, IEEE, Kuala Lumpur, Malaysia, 2008, pp. 1363–1366.
- [25] R. Heradio, H. Perez-Morago, D. Fernandez-Amoros, F.J. Cabrerizo, E. Herrera-Viedma, A bibliometric analysis of 20 years of research on software product lines, *Inf. Softw. Technol.* 72 (2016) 1–15.
- [26] L. Chen, M.A. Babar, A systematic review of evaluation of variability management approaches in software product lines, *Inf. Softw. Technol.* 53 (4) (2011) 344–362.
- [27] A. Classen, P. Heymans, P.Y. Schobbens, What’s in a feature: A requirements engineering perspective, in: *International Conference on Fundamental Approaches To Software Engineering*, Springer, Berlin, Heidelberg, Berlin, Germany, 2008, pp. 16–30.
- [28] K.C. Kang, S.G. Cohen, J.A. Hess, W.E. Novak, A.S. Peterson, *Feature-Oriented Domain Analysis (FODA) Feasibility Study*, Tech. Rep. CMU/SEI-90-TR-21, Carnegie-Mellon Univ Pittsburgh Pa Software Engineering Inst, 1990.
- [29] T. Asikainen, T. Mannisto, T. Soininen, A unified conceptual foundation for feature modelling, in: *10th International Software Product Line Conference, SPLC’06*, IEEE, Baltimore, Maryland, 2006, pp. 31–40.
- [30] H. Eyal-Salman, A.D. Seriai, C. Dony, Feature-to-code traceability in a collection of software variants: Combining formal concept analysis and information retrieval, in: *14th Int. Conf. on Information Reuse & Integration*, IEEE, San Francisco, California, USA, 2013, pp. 209–216.
- [31] M. Raatikainen, J. Tiihonen, T. Männistö, Software product lines and variability modeling, *J. Syst. Softw.* 149 (2019) 485–510.
- [32] J.M. Horcas, J.A. Galindo, R. Heradio, D. Fernandez-Amoros, D. Benavides, A Monte Carlo tree search conceptual framework for feature model analyses, *J. Syst. Softw.* 195 (2023) 111551.
- [33] I.J.S. WG4, ISO/IEC 26550:2015 Software and Systems Engineering. Reference Model for Product Line Engineering and Management, Tech. Rep., International Organization for Standardization (ISO), 2015.
- [34] I.J.S. WG4, ISO/IEC 26580:2021 software and systems engineering. Methods and tools for the feature-based approach to software and systems product line engineering, Tech. Rep., International Organization for Standardization (ISO), 2021.
- [35] L. Etxeberria, G. Sagardui, Evaluation of quality attribute variability in software product families, in: *15th Annual IEEE Int. Conf. and Workshop on the Engineering of Computer Based Systems*, IEEE, 2008, pp. 255–264.
- [36] R. Queiroz, T. Berger, K. Czarnecki, Towards predicting feature defects in software product lines, in: *Proceedings of the 7th International Workshop on Feature-Oriented Software Development*, ACM, 2016, pp. 58–62.
- [37] M. Kowal, S. Ananieva, T. Thüm, Explaining anomalies in feature models, in: *ACM SIGPLAN Notices*, vol. 52-3, ACM, 2016, pp. 132–143.
- [38] A.P. Allian, E. Oliveira Jr., R. Capilla, E.Y. Nakagawa, Have variability tools fulfilled the needs of the software industry? *J. UCS* 26 (10) (2020) 1282–1311.
- [39] K. Feichtinger, K. Meixner, R. Rabiser, S. Biffl, Variability transformation from industrial engineering artifacts: An example in the cyber-physical production systems domain, in: *Proc. of 24th ACM Int. Systems and Soft. Product Line Conference*, ACM, Montreal Quebec, Canada, 2020, pp. 65–73.
- [40] C. Cares, D. Lühr, S. Mora, C. Navarro, L. Olivares, S. Sepúlveda, G. Vidal, Architecting autonomous underwater vehicles by adapting software product lines, in: *Conference on Integrated Computer Technologies in Mechanical Engineering-Synergetic Engineering*, Springer, Charm, Springer Nature Switzerland, Kharkiv, Ukraine, 2022, pp. 719–730.
- [41] V. Bischoff, K. Farias, L.J. Gonçalves, J.L.V. Barbosa, Integration of feature models: A systematic mapping study, *Inf. Softw. Technol.* 105 (2019) 209–225.
- [42] M.A. Nielsen, L.L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010.
- [43] A. Steane, Quantum computing, *Rep. Progr. Phys.* 61 (2) (1998) 117–173.
- [44] V. Vedral, *Introduction to Quantum Information Science*, Oxford University Press, 2006.
- [45] R. Horodecki, P. Horodecki, M. Horodecki, K. Horodecki, Quantum entanglement, *Rev. Modern Phys.* 81 (2) (2009) 865.
- [46] J.P. Dowling, G.J. Milburn, Quantum technology: the second quantum revolution, *Phil. Trans. R. Soc. A* 361 (1809) (2003) 1655–1674.
- [47] N.P. De Leon, K.M. Itoh, D. Kim, K.K. Mehta, T.E. Northup, H. Paik, B.S. Palmer, N. Samarth, S. Sangtawesin, D.W. Steuerman, Materials challenges and opportunities for quantum computing hardware, *Science* 372 (2021) eabb2823, [http://dx.doi.org/10.1126/science.abb2823](https://doi.org/10.1126/science.abb2823).
- [48] D. Ristè, M.P. Da Silva, C.A. Ryan, A.W. Cross, A.D. Córcoles, J.A. Smolin, J.M. Gambetta, J.M. Chow, B.R. Johnson, Demonstration of quantum advantage in machine learning, *npj Quantum Inf.* 3 (1) (2017) 16.
- [49] A.S. Bhatia, A. Kumar, Quadratic sieve factorization quantum algorithm and its simulation, 2020, [arXiv preprint arXiv:2005.11668](https://arxiv.org/abs/2005.11668).
- [50] S. Khairy, R. Shaydulin, L. Cincio, Y. Alexeev, P. Balaprakash, Learning to optimize variational quantum circuits to solve combinatorial problems, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34-03, 2020, pp. 2367–2375.
- [51] M. Cerezo, A. Arrasmith, R. Babbush, S.C. Benjamin, S. Endo, K. Fujii, J.R. McClean, K. Mitarai, X. Yuan, L. Cincio, et al., Variational quantum algorithms, *Nat. Rev. Phys.* 3 (9) (2021) 625–644.

- [52] M.A. Serrano, J.A. Cruz-Lemus, R. Pérez-Castillo, M. Piattini, Quantum software components and platforms: Overview and quality assessment, *ACM Comput. Surv.* 55 (8) (2022) <http://dx.doi.org/10.1145/3548679>.
- [53] M. Stefano, et al., Software engineering for quantum programming, *ACM Comput. Surv.* 55 (7) (2022) 1–35, <http://dx.doi.org/10.1145/3543854>.
- [54] F. Gemeinhardt, A. Garmendia, M. Wimmer, Towards model-driven quantum software engineering, in: 2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering, Q-SE, IEEE, 2021, pp. 13–15.
- [55] S. Boyn, et al., Algorithm and simulation for all-electron correlation, *Phys. Rev. B* 104 (5) (2021) 054503, <http://dx.doi.org/10.1103/PhysRevB.104.054503>.
- [56] R. Pérez-Castillo, M.A. Serrano, M. Piattini, Software modernization to embrace quantum technology, *Adv. Eng. Softw.* 151 (2021) 102933.
- [57] B. Weder, J. Barzen, F. Leymann, M. Zimmermann, Hybrid Quantum Applications Need Two Orchestrations in Superposition: A Software Architecture Perspective, in: Proceedings of the 18th IEEE International Conference on Web Services, ICWS 2021, IEEE, 2021, pp. 1–13, <http://dx.doi.org/10.1109/ICWS53863.2021.00015>.
- [58] F. Leymann, J. Barzen, M. Falkenthal, D. Vietz, B. Weder, K. Wild, Quantum in the cloud: application potentials and research opportunities, 2020, arXiv preprint [arXiv:2003.06256](https://arxiv.org/abs/2003.06256).
- [59] S. Magaz-Romero, E. Mosqueira-Rey, D. Alvarez-Estevéz, V. Moret-Bonillo, Quantum factory method: A software engineering approach to deal with incompatibilities in quantum libraries, in: International Conference on Computational Science, Springer, 2023, pp. 64–71.
- [60] I. Gheorghe-Pop, N. Tcholtchev, T. Ritter, M. Hauswirth, Quantum DevOps: Towards reliable and applicable NISQ quantum computing, in: 2020 IEEE Globecom Workshops, GC Wkshps, IEEE, Taipei, Taiwan, 2020, pp. 1–6, <http://dx.doi.org/10.1109/GCWkshps50303.2020.9367411>.
- [61] C. O'Meara, M. Fernández-Campoamor, G. Cortiana, J. Bernabé-Moreno, Quantum software architecture blueprints for the cloud: Overview and application to peer-2-peer energy trading, in: 2023 IEEE Conference on Technologies for Sustainability, SusTech, IEEE, 2023, pp. 191–198.
- [62] J. Alvarado-Valiente, J. Romero-Álvarez, E. Moguel, J. García-Alonso, Quantum web services orchestration and management using DevOps techniques, in: International Conference on Web Engineering, Springer, 2023, pp. 389–394.
- [63] O. Di Matteo, S. Nunez-Corrales, M. Stechly, S.P. Reinhardt, T. Mattson, An abstraction hierarchy toward productive quantum programming, 2024, arXiv preprint [arXiv:2405.13918](https://arxiv.org/abs/2405.13918).
- [64] S. Sepúlveda, A. Cravero, G. Fonseca, L. Antonelli, Systematic review on requirements engineering in quantum computing: Insights and future directions, *Electronics* 13 (15) (2024) 2989.
- [65] H. Safi, K. Wintersperger, W. Mauerer, Influence of HW-SW-co-design on quantum computing scalability, 2023, [arXiv:2306.04246](https://arxiv.org/abs/2306.04246).
- [66] S. Ali, T. Yue, R. Abreu, When software engineering meets quantum computing, *Commun. ACM* 65 (4) (2022) 84–88.
- [67] O. Di Matteo, On the need for effective tools for debugging quantum programs, 2024, arXiv preprint [arXiv:2402.09547](https://arxiv.org/abs/2402.09547).
- [68] K.J. Ah, Case study of software product line engineering in insurance product, in: Proceedings of the 14th International Conference on Software Product Lines: Going beyond, SPLC '10, Springer-Verlag, Berlin, Heidelberg, 2010, p. 495.
- [69] Y. Wu, X. Peng, W. Zhao, Architecture evolution in software product line: An industrial case study, in: K. Schmid (Ed.), Top Productivity Through Software Reuse, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 135–150.
- [70] J. Ferreira, P. da Mota, A. Paulo, E. Santana de Almeida, S. Romero de Lemos, Adopting software product lines: A systematic mapping study, in: 15th Annual Conference on Evaluation & Assessment in Software Engineering, EASE 2011, IET, Durham, 2011, pp. 11–20, <http://dx.doi.org/10.1049/ic.2011.0002>.
- [71] D. Amalfitano, V. De Simone, A. Fasolino, M. Lubrano, S. Scala, Introducing software product lines in model-based design processes: An industrial experience, in: 2016 13th Working IEEE/IFIP Conference on Software Architecture, WICSA, IEEE, Venice, Italy, 2016, pp. 287–290, <http://dx.doi.org/10.1109/WICSA.2016.36>.
- [72] K. Ignaim, J. Fernandes, An industrial case study for adopting software product lines in automotive industry: An evolution-based approach for software product lines (EVOA-SPL), in: Proceedings of the 23rd International Systems and Software Product Line Conference-Volume B, ACM, New York, United States, 2019, pp. 183–190.
- [73] M. Esparguel, S. Sepúlveda, E. Monsalve, FMxx: A proposal for the creation, management and review of feature models in software product lines, in: 2017 36th International Conference of the Chilean Computer Science Society, SCCS, IEEE, Arica, Chile, 2017, pp. 1–7.
- [74] M. Setyautami, H. Fadhilillah, D. Adianto, I. Affan, A. Azurat, Variability management: Re-engineering microservices with delta-oriented software product lines, in: Proceedings of the 24th ACM Conference on Systems and Software Product Line: Volume a-Volume a, ACM, Montreal Quebec Canada, 2020, pp. 1–6.
- [75] O. Aguayo, S. Sepúlveda, Variability management in dynamic software product lines for self-adaptive systems—a systematic mapping, *Appl. Sci.* 12 (20) (2022) 10240.
- [76] F. Wilhelm, R. Steinwandt, B. Langenberg, P. Liebermann, A. Messinger, P. Schuhmacher, A. Misra-Spieldenner, Status of Quantum Computer Development, V1.2., Tech. Rep., Federal Office for Information Security, 2020.
- [77] J. García-Alonso, J. Rojo, D. Valencia, E. Moguel, J. Berrocal, J.M. Murillo, Quantum software as a service through a quantum API gateway, *IEEE Internet Comput.* 26 (1) (2021) 34–41.
- [78] J.L. Hevia, G. Peterssen, M. Piattini, qSOA[®]: Dynamic integration for hybrid quantum/classical software systems, *J. Syst. Softw.* 214 (2024) 112061.
- [79] R. Correll, S.J. Weinberg, F. Sanches, T. Ide, T. Suzuki, Quantum neural networks for a supply chain logistics application, 2022, [arXiv:2212.00576](https://arxiv.org/abs/2212.00576).
- [80] S.J. Weinberg, F. Sanches, T. Ide, K. Kamiya, R. Correll, Supply chain logistics with quantum and classical annealing algorithms, 2022, [arXiv:2205.04435](https://arxiv.org/abs/2205.04435).
- [81] D. Knoll, J. Waldmann, G. Reinhart, Developing an internal logistics ontology for process mining, *Procedia CIRP* 79 (2019) 427–432, <http://dx.doi.org/10.1016/j.procir.2019.02.116>, URL <https://www.sciencedirect.com/science/article/pii/S2212827119302331>.
- [82] H. Hartmann, T. Trew, Using feature diagrams with context variability to model multiple product lines for software supply chains, in: 2008 12th International Software Product Line Conference, IEEE, Limerick, Ireland, 2008, pp. 12–21, <http://dx.doi.org/10.1109/SPLC.2008.15>.
- [83] J. Ariño, R. Palacios, Adiabatic quantum computing for logistic transport optimization, 2023, [arXiv:2301.07691](https://arxiv.org/abs/2301.07691).
- [84] J. Ariño, R. Palacios, Adiabatic quantum computing impact on transport optimization in the last-mile scenario, *Front. Comput. Sci.* 5 (2023) <http://dx.doi.org/10.3389/fcomp.2023.1294564>, URL <https://www.frontiersin.org/articles/10.3389/fcomp.2023.1294564>.
- [85] K. B., D. Brand, A. Ceschini, C. Chou, R.-H. L., K. Pandya, A. Summer, A review on quantum approximate optimization algorithm and its variants, 2023, [arXiv:2306.09198](https://arxiv.org/abs/2306.09198).
- [86] S. d'Souza, Intelligent Supply Chain Management Using Quantum, Research Report, TCS, 2022, URL <https://hal.science/hal-03740772>.
- [87] ISO/IEC/IEEE, ISO/IEC/IEEE 15288:2015 International Standard - Systems and software engineering—System life cycle processes, Tech. Rep., International Organization for Standardization (ISO), 2015.
- [88] C.W. Krueger, P.C. Clements, Second generation systems and software product line engineering, in: Proceedings of the 19th International Conference on Software Product Line, SPLC '15, Association for Computing Machinery, New York, NY, USA, 2015, pp. 388–389, <http://dx.doi.org/10.1145/2791060.2798047>.
- [89] P. Clements, S. Gregg, C. Krueger, J. Lanman, J. Rivera, R. Scharadin, J. Shepherd, A. Winkler, Second generation product line engineering takes hold in the dod, *Crosstalk J. Def. Softw. Eng. USAF Softw. Technol. Support Center* (2013).
- [90] T. Tomesh, M. Martonosi, Quantum codesign, *IEEE Micro* 41 (5) (2021) 33–40, <http://dx.doi.org/10.1109/MM.2021.3094461>.
- [91] G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, Y. Xie, On the co-design of quantum software and hardware, in: NANOCOM '21: Proceedings of the Eight Annual ACM International Conference on Nanoscale Computing and Communication, NANOCOM '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–7, <http://dx.doi.org/10.1145/3477206.3477464>.